

Práctico 4

Planificación de uso de CPU

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento presenta la resolución integral del Práctico 4 de la materia Sistemas Operativos (UNRC, 2026), centrado en la *planificación de uso de CPU*. Se estudian seis ejercicios que recorren los algoritmos clásicos: *First-Come First-Served* (FCFS), *Shortest Job First* (SJF), *Round Robin* (RR), planificación por *prioridades* estáticas y dinámicas y *Multilevel Feedback Queues* (MLF), más una discusión sobre *processor affinity* vs. *load balancing* en sistemas SMP. Cada ejercicio se justifica con la teoría del capítulo *Planificación de uso de CPU* de las Notas del curso (Marcelo Arroyo) y se acompaña con tablas de procesos, diagramas de Gantt y cálculos paso a paso de los tiempos de espera, turnaround y respuesta.

Índice

1. Introducción	1
2. Ejercicio 1 — FCFS y SJF: tiempo de espera promedio	2
2.1. 1.a FCFS (FIFO) con los arribos dados	2
2.2. 1.b SJF (no preemptivo, todos en $t = 0$)	3
3. Ejercicio 2 — Round Robin y comparación con FCFS y SJF	3
3.1. 2.a Turnaround de cada proceso	4
3.2. 2.b Comparación con FCFS y SJF	4
4. Ejercicio 3 — Prioridades dinámicas (estilo 4.3BSD)	4
4.1. 3.a CPU-bound vs. I/O-bound	5
4.2. 3.b Objetivo del planificador	5
5. Ejercicio 4 — Planificación por prioridades estáticas	5
6. Ejercicio 5 — Multilevel Feedback de tres niveles	6
7. Ejercicio 6 — Processor affinity vs. load balancing en SMP	8
8. Conclusiones	9

1. Introducción

El *scheduler* de CPU es el subsistema del SO que decide *cuándo*, *dónde* y *por cuánto tiempo* cada tarea va a ejecutar. La política que adopta depende del tipo de sistema y de la carga de trabajo: en sistemas *batch* se busca minimizar el tiempo medio de espera; en *time-sharing*, acotar el tiempo de respuesta; en *multiprocesadores SMP*, equilibrar la carga sin romper la afinidad de caché. El capítulo de las Notas resume los criterios habituales:

- Maximizar la utilización de la CPU.

- **Justicia (*fairness*):** tiempos parejos para cada tarea.
- **Throughput:** procesos completados por unidad de tiempo.
- **Tiempo de retorno (*turnaround*):** tiempo total que un proceso pasa en el sistema, desde que arribó hasta que terminó.
- **Tiempo de espera:** tiempo en estado **READY** sin ejecutar (CPU asignada a otro).
- **Tiempo de respuesta:** latencia desde que llega un requerimiento hasta que el sistema empieza a procesarlo.

Varios de estos criterios se contraponen entre sí: maximizar *throughput* puede deteriorar el tiempo de respuesta; balancear la carga puede afectar la afinidad de caché. Por eso no existe “el” mejor algoritmo, sino el más adecuado para cada escenario.

Las fórmulas que se usan a lo largo del práctico son las estándar:

$$\begin{aligned} \text{Espera}_i &= \text{Inicio}_i - \text{Arribo}_i \quad (\text{para algoritmos no preemptivos sin I/O}) \\ \text{Turnaround}_i &= \text{Fin}_i - \text{Arribo}_i \\ \text{Respuesta}_i &= \text{PrimerInicio}_i - \text{Arribo}_i \end{aligned}$$

En la versión preemptiva con interrupciones por timer, el tiempo de espera se calcula como $\text{Espera}_i = \text{Turnaround}_i - \text{CPU}_i - \text{WAIT}_i$.

2. Ejercicio 1 — FCFS y SJF: tiempo de espera promedio

Proceso	Tiempo de creación (arribo)	Ráfaga de CPU (μs)
P1	0.0	8
P2	0.5	3
P3	3.0	5

Se pide el tiempo de espera promedio bajo (1) FCFS con los tiempos de arribo dados y (2) SJF *asumiendo que los tres arriban en $t = 0$.*

2.1. 1.a FCFS (FIFO) con los arribos dados

FCFS es el algoritmo más simple: cola FIFO, no preemptivo. Se otorga la CPU al primero de la cola y no se la quita hasta que termine. El orden de arribo es P1, P2, P3.

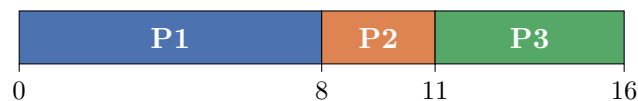


Diagrama de Gantt — FCFS

Proceso	Arribo	Inicio	Fin	Espera
P1	0.0	0	8	0.0
P2	0.5	8	11	7.5
P3	3.0	11	16	8.0

$$\bar{W}_{\text{FCFS}} = \frac{0 + 7.5 + 8}{3} = \frac{15.5}{3} \approx 5.17.$$

Observación. P1 es la ráfaga más larga y llega primero, así que arrastra la espera de los demás (*convoy effect*). Es exactamente la desventaja que la teoría le adjudica a FCFS: “comúnmente produce tiempos promedio de espera grandes”.

2.2. 1.b SJF (no preemptivo, todos en $t = 0$)

Bajo el supuesto del enunciado, los tres procesos están disponibles desde $t = 0$. SJF elige siempre la ráfaga más corta entre los READY: P2 (3) primero, P3 (5) después, P1 (8) al final.



Diagrama de Gantt — SJF

Proceso	Arribo	Inicio	Fin	Espera
P2	0	0	3	0
P3	0	3	8	3
P1	0	8	16	8

$$\overline{W}_{\text{SJF}} = \frac{0 + 3 + 8}{3} = \frac{11}{3} \approx 3.67.$$

Comparación

Algoritmo	Orden	Espera promedio
FCFS	P1 - P2 - P3	5.17
SJF (todos en $t = 0$)	P2 - P3 - P1	3.67

SJF da menor tiempo de espera promedio. La razón es la propiedad clásica del algoritmo: *dado un conjunto fijo de ráfagas conocidas, ejecutar primero las más cortas minimiza la espera promedio*. Esta propiedad de optimalidad es la motivación histórica de SJF en sistemas *batch* con *long-term schedulers*, donde el usuario declara el tiempo estimado del job.

3. Ejercicio 2 — Round Robin y comparación con FCFS y SJF

Para los procesos del Ejercicio 1 se aplica **Round Robin** preemptivo con quantum $q = 2$. Se mantiene una cola FIFO de **RUNNABLE**: cuando un proceso llega o vuelve por consumir su quantum se encola al final; el scheduler toma siempre el primero de la cola.

Convención de empates. Cuando un nuevo arribo y un desalojo ocurren al mismo instante, primero se encola el recién llegado y luego el desalojado. Es la convención habitual de los textos clásicos (Silberschatz, Tanenbaum).

Traza paso a paso

Notación: la cola se escribe [cabeza ... cola] y entre paréntesis aparece la ráfaga remanente.

Intervalo	CPU	Evento al final del intervalo	Cola READY
0–2	P1	$t = 0.5$: arriba P2; $t = 2$: P1 fin de quantum (rem. 6)	[P2(3), P1(6)]
2–4	P2	$t = 3$: arriba P3; $t = 4$: P2 fin de quantum (rem. 1)	[P1(6), P3(5), P2(1)]
4–6	P1	$t = 6$: P1 fin de quantum (rem. 4)	[P3(5), P2(1), P1(4)]
6–8	P3	$t = 8$: P3 fin de quantum (rem. 3)	[P2(1), P1(4), P3(3)]
8–9	P2	$t = 9$: P2 termina (ráfaga era 1)	[P1(4), P3(3)]
9–11	P1	$t = 11$: P1 fin de quantum (rem. 2)	[P3(3), P1(2)]
11–13	P3	$t = 13$: P3 fin de quantum (rem. 1)	[P1(2), P3(1)]
13–15	P1	$t = 15$: P1 termina	[P3(1)]
15–16	P3	$t = 16$: P3 termina	[]

Verificación: la CPU está ocupada de 0 a 16 sin huecos y $8 + 3 + 5 = 16$. Cuadra.

Diagrama de Gantt



3.1. 2.a Turnaround de cada proceso

$T_i = \text{Fin}_i - \text{Arribo}_i$:

Proceso	Arribo	Fin	Turnaround
P1	0.0	15	15.0
P2	0.5	9	8.5
P3	3.0	16	13.0

$$\bar{T}_{\text{RR}} = \frac{15 + 8.5 + 13}{3} = \frac{36.5}{3} \approx 12.17.$$

3.2. 2.b Comparación con FCFS y SJF

Los tres algoritmos no parten de las mismas hipótesis: FCFS y RR usan los arribos originales; SJF (parte b del Ejercicio 1) asume todos en $t = 0$. Aún así, la comparación es ilustrativa:

Proceso	T FCFS	T SJF	T RR
P1	8.0	16	15.0
P2	10.5	3	8.5
P3	13.0	8	13.0
Promedio	10.50	9.00	12.17

Lectura:

- **SJF** es el mejor en turnaround promedio (9.0): la propiedad de optimalidad garantiza que ejecutar primero las ráfagas más cortas minimiza la espera promedio.
- **FCFS** queda intermedio (10.5). Sufre el *convoy effect* porque P1 llega primero y arrastra a los demás, pero cada proceso ejecuta de un solo tirón.
- **RR** es el peor en promedio (12.17): el quantum chico fragmenta la ejecución de P1 en cuatro pedazos, pero a cambio P2 mejora notablemente ($10.5 \rightarrow 8.5$) porque no queda atascada detrás de la ráfaga más larga.

Esto es justamente el compromiso clásico que la teoría señala: RR no está pensado para minimizar turnaround promedio sino para acotar el *tiempo de respuesta* en sistemas *time-sharing*. Con quantum tendiendo al infinito, RR converge a FCFS; con quantum muy chico, el overhead de cambios de contexto domina.

4. Ejercicio 3 — Prioridades dinámicas (estilo 4.3BSD)

Un planificador recalcula cada segundo la prioridad de cada proceso con la fórmula

$$\text{priority} = \frac{\text{recent_cpu_usage}()}{2} + 60$$

donde *a mayor valor, menor prioridad*. Tres procesos tienen usos recientes de CPU de 30, 15 y 10 respectivamente.

Cálculo de prioridades

Proceso	recent_cpu	priority = recent_cpu/2 + 60	Posición
P1	30	30/2 + 60 = 75	menor prioridad
P2	15	15/2 + 60 = 67.5	intermedia
P3	10	10/2 + 60 = 65	mayor prioridad

Orden de prioridad (de más alta a más baja): $P_3 \succ P_2 \succ P_1$.

4.1. 3.a CPU-bound vs. I/O-bound

Un proceso **CPU-bound** ejecuta ráfagas largas: gasta su quantum sin bloquearse, así que su contador `recent_cpu_usage` crece rápido. Su `priority` numérica sube y, como mayor valor implica menor prioridad efectiva, **el algoritmo lo posterga**. P1 (uso 30, prioridad 75) es el ejemplo en la tabla.

Un proceso **I/O-bound** se bloquea seguido en `read` o `write` y rara vez consume el quantum entero. Su contador queda chico, su `priority` se mantiene cerca del piso (60), y el algoritmo **lo favorece** cuando vuelve de I/O. P3 (uso 10, prioridad 65) lo ilustra.

El término “+60” funciona como piso fijo (ningún proceso baja de 60) y el “/2” amortigua el crecimiento, evitando que un pico breve de uso castigue desproporcionadamente al proceso. La fórmula provista es exactamente la receta del scheduler clásico de **4.3BSD UNIX**: *multilevel feedback* con *decay* exponencial del `recent_cpu`.

4.2. 3.b Objetivo del planificador

Dos objetivos complementarios:

- **Mejor responsiveness para procesos interactivos:** editores, shells y servicios atendiendo *requests* son típicamente I/O-bound. Que el scheduler les dé prioridad alta cuando vuelven de un bloqueo significa baja latencia entre el evento y su procesamiento.
- **Mejor uso de los dispositivos de I/O:** si los procesos I/O-bound se planifican rápido mantienen los dispositivos ocupados; los CPU-bound usan la CPU mientras los otros están bloqueados, sin perjudicar el throughput global.

La penalización a los CPU-bound **no es permanente**: como `recent_cpu_usage` solo refleja la historia reciente, un proceso CPU-bound que deja de demandar CPU verá caer su prioridad numérica y volverá a ser elegible. Es exactamente el rol que la teoría le adjudica al *aging*.

En una línea: el planificador busca un **compromiso entre utilización de CPU, throughput y tiempo de respuesta**, que la teoría identifica como el objetivo central de los *short-term schedulers* en sistemas *time-sharing*.

5. Ejercicio 4 — Planificación por prioridades estáticas

Para los procesos del Ejercicio 1 (todos arriban en $t = 0$), con prioridades:

Proceso	CPU	Prioridad
P1	8	1
P2	3	3
P3	5	2

Convención. El enunciado no fija si menor número implica mayor o menor prioridad. Se adopta la convención más extendida en bibliografía clásica (Silberschatz, Tanenbaum) y la que

usa Linux en el rango *real-time*: **menor número** $\rightarrow$ **mayor prioridad**. Bajo esta convención el orden decreciente es $P_1 \succ P_3 \succ P_2$.

Preemptivo vs. no preemptivo. Como los tres llegan en $t = 0$ y nadie se incorpora después, no hay diferencia entre las dos versiones: una vez decidido el orden, ningún arribo posterior lo puede alterar.

Diagrama de Gantt

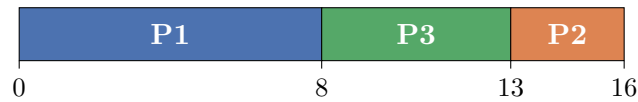


Diagrama de Gantt — Prioridades estáticas (menor número $\rightarrow$ mayor prioridad)

Proceso	Inicio	Fin	Espera	Turnaround
P1	0	8	0	8
P3	8	13	8	13
P2	13	16	13	16

Promedios: $\bar{W} = (0 + 8 + 13)/3 = 7$; $\bar{T} = (8 + 13 + 16)/3 \approx 12.33$.

Observaciones

1. La secuencia depende *exclusivamente* del número de prioridad. Aquí P1 (mayor prioridad) coincide con la ráfaga más larga, lo que perjudica el promedio respecto de SJF (orden P2-P3-P1).
2. La planificación por prioridades **no garantiza optimalidad** en tiempo de espera ni en turnaround promedio: solo respeta la importancia relativa que el sistema asigna a cada proceso. Es una decisión política, no temporal.
3. **Riesgo de starvation**: si llegasen continuamente procesos con prioridad 1 o 2, P2 (prioridad 3) podría postergarse indefinidamente. La solución estándar —mencionada explícitamente por la teoría— es el *aging*: incrementar gradualmente la prioridad de procesos que llevan mucho tiempo esperando.

Si la convención fuera “mayor número $\rightarrow$ mayor prioridad”

El orden sería $P_2 \succ P_3 \succ P_1$:



Diagrama de Gantt — convención inversa

Promedios: $\bar{W} \approx 3.67$, $\bar{T} = 9$. Coincide numéricamente con SJF en el Ejercicio 1, porque casualmente las “prioridades altas” coinciden con las ráfagas más cortas.

6. Ejercicio 5 — Multilevel Feedback de tres niveles

Se pide describir paso a paso un scheduler MLF con tres niveles L_0 , L_1 , L_2 y *quantums* 3, 2 y 1 ms respectivamente, sobre las tareas:

Tarea	Comportamiento (línea temporal)
T1	CPU = 5 ms ; WAIT = 5 ms ; CPU = 1 ms
T2	CPU = 12 ms
T3	CPU = 1 ms ; WAIT = 2 ms ; CPU = 1 ms

Las tres ingresan a L_0 en $t = 0$ en el orden T1, T2, T3.

Reglas del MLF

- Niveles ordenados por prioridad: L_0 es el más prioritario, L_2 el menos.
- El scheduler ejecuta siempre al primero del nivel más alto *no vacío*.
- Dentro de cada nivel se aplica RR con su propio quantum.
- **Demote**: si un proceso *consume todo el quantum* se considera CPU-bound y baja un nivel. Si ya está en L_2 se reencola en el mismo L_2 .
- **Sin demote**: si un proceso *cede voluntariamente* (bloqueo en I/O) o *termina* antes del quantum, se considera interactivo y se queda en su nivel.
- **Preemption entre niveles**: si mientras un proceso ejecuta llega/desperta otro de nivel superior, el primero es desalojado y reencolado al final de su nivel actual.
- Convención en empates: (a) si llegada y desalojo coinciden en el tiempo, primero se encola la nueva; (b) si la ráfaga termina justo al expirar el quantum, prevalece la lectura “I/O” (no demote); (c) un proceso que despierta de WAIT regresa al mismo nivel donde estaba antes del bloqueo.

Tabla temporal

Notación: $T_i:n$ significa “tarea i con n ms pendientes de CPU en su ráfaga actual”. $\langle \rangle$ es cola vacía.

Time	CPU	T1	T2	T3	L_0	L_1	L_2
0	—	RDY	RDY	RDY	$\langle T1:5, T2:12, T3:1 \rangle$	$\langle \rangle$	$\langle \rangle$
0–3	T1	RUN	RDY	RDY	$\langle T2:12, T3:1 \rangle$	$\langle \rangle$	$\langle \rangle$
3	demote T1	RDY	RDY	RDY	$\langle T2:12, T3:1 \rangle$	$\langle T1:2 \rangle$	$\langle \rangle$
3–6	T2	RDY	RUN	RDY	$\langle T3:1 \rangle$	$\langle T1:2 \rangle$	$\langle \rangle$
6	demote T2	RDY	RDY	RDY	$\langle T3:1 \rangle$	$\langle T1:2, T2:9 \rangle$	$\langle \rangle$
6–7	T3	RDY	RDY	RUN	$\langle \rangle$	$\langle T1:2, T2:9 \rangle$	$\langle \rangle$
7	T3 $\rightarrow$ WAIT	RDY	RDY	WAIT	$\langle \rangle$	$\langle T1:2, T2:9 \rangle$	$\langle \rangle$
7–9	T1	RUN	RDY	WAIT	$\langle \rangle$	$\langle T2:9 \rangle$	$\langle \rangle$
9	T1 $\rightarrow$ WAIT; T3 wakeup	WAIT	RDY	RDY	$\langle T3:1 \rangle$	$\langle T2:9 \rangle$	$\langle \rangle$
9–10	T3	WAIT	RDY	RUN	$\langle \rangle$	$\langle T2:9 \rangle$	$\langle \rangle$
10	T3 termina	WAIT	RDY	TERM	$\langle \rangle$	$\langle T2:9 \rangle$	$\langle \rangle$
10–12	T2	WAIT	RUN	TERM	$\langle \rangle$	$\langle \rangle$	$\langle \rangle$
12	demote T2	WAIT	RDY	TERM	$\langle \rangle$	$\langle \rangle$	$\langle T2:7 \rangle$
12–13	T2	WAIT	RUN	TERM	$\langle \rangle$	$\langle \rangle$	$\langle \rangle$
13	T2 reencola en L_2	WAIT	RDY	TERM	$\langle \rangle$	$\langle \rangle$	$\langle T2:6 \rangle$
13–14	T2	WAIT	RUN	TERM	$\langle \rangle$	$\langle \rangle$	$\langle \rangle$
14	T1 wakeup $\rightarrow L_1$; T2 desalojado	RDY	RDY	TERM	$\langle \rangle$	$\langle T1:1 \rangle$	$\langle T2:5 \rangle$
14–15	T1	RUN	RDY	TERM	$\langle \rangle$	$\langle \rangle$	$\langle T2:5 \rangle$
15	T1 termina	TERM	RDY	TERM	$\langle \rangle$	$\langle \rangle$	$\langle T2:5 \rangle$
15–20	T2	TERM	RUN	TERM	$\langle \rangle$	$\langle \rangle$	$\langle T2:4..0 \rangle$
20	T2 termina	TERM	TERM	TERM	$\langle \rangle$	$\langle \rangle$	$\langle \rangle$

Verificación: la CPU está ocupada de 0 a 20 ms sin huecos. Las ráfagas suman $T_1 = 6$, $T_2 = 12$, $T_3 = 2$, total 20 ms.

Justificación de los pasos clave

- $t = 0-3$: T1 toma CPU en L_0 (quantum 3). Su ráfaga era 5; consume el quantum, le quedan 2. Demote a L_1 .
- $t = 3-6$: T2 corre los 3 ms del quantum de L_0 . Le quedan 9. Demote a L_1 .
- $t = 6-7$: T3 obtiene CPU en L_0 y completa su ráfaga de 1 ms (cede voluntariamente al pasar a WAIT por 2 ms). **No demote**: cuando despierte vuelve a L_0 .
- $t = 7-9$: con L_0 vacío, el scheduler baja a L_1 y elige T1. T1 corre los 2 ms del quantum de L_1 y *justo* completa su primera ráfaga de 5 ms (3 en L_0 + 2 en L_1); pasa a WAIT por 5 ms. Por la convención adoptada (ráfaga termina exactamente en el quantum $\rightarrow$ I/O), T1 no demote a L_2 : cuando despierte volverá a L_1 . Simultáneamente, T3 despierta y vuelve a L_0 .

- $t = 9\text{--}10$: T3 obtiene CPU (preferencia de L_0 sobre L_1), corre su última ráfaga de 1 ms y termina.
- $t = 10\text{--}13$: T2 ejecuta en L_1 (2 ms), demote a L_2 , ejecuta otro ms en L_2 . En cada quantum agotado en L_2 se reencola en el mismo nivel.
- $t = 13\text{--}14$: T2 sigue en L_2 . Justo al expirar este quantum, T1 despierta y entra a L_1 .
- $t = 14\text{--}15$: T1 obtiene CPU (preferencia de L_1 sobre L_2), corre 1 ms y termina su última ráfaga.
- $t = 15\text{--}20$: T2 queda sola y completa los 5 ms restantes en L_2 (cinco quantums consecutivos de 1 ms).

Métricas resultantes

Tarea	Llegada	Fin	Turnaround	CPU usada	Espera
T1	0	15	15	6	4
T2	0	20	20	12	8
T3	0	10	10	2	6

Lectura. T3 (interactivo, 2 ms de CPU) sale rápido del sistema; T1 (mixta, 6 ms de CPU) termina segunda. T2 (CPU-bound puro, 12 ms) es *la más castigada*: queda atrapada en L_2 y solo termina al final, después de que las otras se hayan ido. Sin embargo **no padece starvation** porque, una vez que el sistema queda libre, T2 obtiene la CPU sin contención. Es exactamente el comportamiento que la teoría atribuye al MLF: *mejorar el tiempo de respuesta de los procesos interactivos castigando a los CPU-bound, sin negarles servicio*.

7. Ejercicio 6 — Processor affinity vs. load balancing en SMP

En sistemas SMP el scheduler debe lograr *processor affinity* y *load balancing* simultáneamente. Se pide explicar por qué estas propiedades se contraponen.

Definiciones

- **Load balancing**: emparejar la carga de trabajo entre todas las CPUs. En cada *context switch* un proceso puede *migrar* de CPU.
- **Processor affinity**: minimizar la migración de procesos entre CPUs. Linux ofrece la syscall `sched_setaffinity` (*hard affinity*) para ligar un proceso a un subconjunto de CPUs.

La teoría explicita el motivo: la migración *afecta al rendimiento porque la memoria caché de la CPU anterior queda invalidada y la caché de la nueva debe volver a llenarse*.

Por qué se contraponen

Las dos propiedades empujan al scheduler en direcciones opuestas:

1. **Load balancing pide migrar.** Si una CPU está sobrecargada (cola READY larga) y otra está libre, lo correcto *desde el balance* es mover algún proceso a la libre. Sin migración, una CPU queda al 100 % mientras otra desperdicia ciclos en idle.
2. **Processor affinity pide no migrar.** Cada migración descarta el trabajo de *calentamiento de caché* (L1 y L2 son privadas por core). El proceso paga miles de instrucciones con *cache miss* hasta repoblar la nueva caché. También se pierde la huella en la TLB.

El conflicto concreto:

- Si el scheduler **prioriza affinity**, deja de migrar incluso con desbalance: aparece subutilización (una CPU al 100 %, otra al 0 %). El sistema de N CPUs rinde como uno más chico.

- Si el scheduler **prioriza load balancing**, migra seguido para mantener parejas las CPUs. Pero cada migración paga el costo de caché. Si las migraciones son frecuentes el sistema gasta tiempo en repoblar caches en lugar de hacer trabajo útil.

Compromiso práctico

La teoría lo nombra explícitamente: “estos dos objetivos se contraponen, por lo que hay que lograr una solución de compromiso”. Las estrategias clásicas son:

- **Per-CPU runqueue** más un kernel thread dedicado al balanceo (en Linux: el thread `migration` y la rutina `load_balance`). El scheduler local respeta la affinity por defecto; el balanceador estima la carga y migra solo si el desbalance excede un umbral.
- **Soft affinity**: el scheduler trata de no migrar pero puede hacerlo si compensa (default en Linux).
- **Hard affinity**: con `sched_setaffinity` el proceso queda restringido a un conjunto fijo de CPUs.
- **Push vs. pull migration**: una CPU sobrecargada empuja trabajo, o una CPU ociosa lo tira. Coexisten.

El principio común: **migrar solo cuando el desbalance proyectado supera al costo esperado de la migración**. En el extremo NUMA, donde la migración entre nodos suma además el costo de accesos a memoria remota, la affinity gana peso. En arquitecturas *multi-core* con caché L3 compartida entre cores del mismo *socket*, migrar dentro del socket es mucho más barato que entre sockets, y los schedulers modernos (CFS de Linux) son conscientes de esta jerarquía.

Resumen. *Affinity* protege la caché de cada proceso individual; *load balancing* protege la utilización global del multiprocesador. Cada migración gana balance al costo de afinidad y viceversa, así que el scheduler decide cuánto vale cada migración antes de hacerla.

8. Conclusiones

El práctico recorre el menú clásico de algoritmos de planificación de CPU y deja en evidencia los compromisos centrales del diseño:

1. **FCFS** es simple y predecible pero sufre el *convoy effect* cuando una ráfaga larga llega primero (Ej. 1).
2. **SJF** minimiza el tiempo de espera promedio cuando se conocen las ráfagas (Ej. 1), pero es no preemptivo y requiere conocer las duraciones (estimable con promedio exponencial $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$).
3. **Round Robin** acota el tiempo de respuesta a costa de un mayor turnaround promedio cuando hay procesos largos (Ej. 2). El quantum es el parámetro de compromiso.
4. **Prioridades** permiten al sistema imponer una política, pero no garantizan optimalidad temporal y traen el riesgo de *starvation* (Ej. 3 y 4). El *aging* es la solución estándar.
5. **Multilevel Feedback** combina prioridades dinámicas con RR por nivel; es la versión “natural” para sistemas *time-sharing* con cargas heterogéneas, y favorece a los procesos interactivos sin negarle progreso a los CPU-bound (Ej. 5).
6. En **SMP** el scheduler debe equilibrar affinity y load balancing: dos objetivos legítimos pero opuestos, que obligan a tomar decisiones costo-beneficio en cada migración (Ej. 6).

La idea unificadora es que la planificación es siempre un compromiso entre criterios que se contraponen: utilización, *throughput*, tiempo de espera, tiempo de respuesta y *fairness*. Ningún algoritmo gana en todas las métricas a la vez; cada uno tiene su nicho de aplicación. Los schedulers modernos (CFS, EEVDF en Linux desde 2024) son combinaciones sofisticadas de estos principios, sintonizados para cargas reales heterogéneas.